

“트리와 쿼리” 문제 풀이

작성자: 윤교준

부분문제 1

가능한 트리의 형태가 세 가지밖에 없으므로, 조건문으로 쉽게 해결할 수 있다.

부분문제 2

S 에 속한 모든 두 정점쌍에 대하여 S 위에서 연결되어 있는지 여부를 판별하자.

두 정점이 S 위에서 연결되어 있는지는 DFS 등을 이용하면 $\mathcal{O}(N)$ 에 판별할 수 있다.

따라서, 각 쿼리당 $\mathcal{O}(NK^2)$ 에 문제를 해결할 수 있다.

부분문제 3

트리에서 S 에 속한 정점만 남기고 다른 모든 정점을 제거하면, 여러 개의 트리로 이루어진 포레스트(Forest)가 된다.

이 포레스트의 연결 컴포넌트를 “ S 위에서 연결 컴포넌트”라고 하자.

S 의 연결 강도는 S 위에서 연결 컴포넌트 각각의 크기를 통하여 계산할 수 있다.

따라서, S 에 속하는 정점만을 사용하여 DFS를 수행하면, 각 쿼리당 $\mathcal{O}(N)$ 에 문제를 해결할 수 있다.

일반적으로, 정점 집합의 부분 집합 $V' \subseteq V$ 를 방문하는 DFS의 시간 복잡도는 다음과 같다:

$$\sum_{v \in V'} \deg(v)$$

V' 에 속하는 각 정점 v 마다 $\deg(v)$ 개의 간선을 참조하기 때문이다.

이 문제의 경우, 일반적인 DFS를 수행한다면, 시간 복잡도는 다음과 같다:

$$\sum_{v \in S} \deg(v)$$

이 값은 집합의 크기 $|S| = K$ 보다 유의미하게 클 수 있음에 유의하라.

예를 들어, 1번 정점에 다른 모든 정점이 붙어 있는 트리 위에서 $S = \{1, 2, \dots, K\}$ 를 생각하자. $\deg(1) = N - 1$ 이므로 K 의 값에 무관하게, degree의 합이 $\mathcal{O}(N)$ 임을 확인할 수 있다.

부분문제 4

Union-find 기법을 응용하자.

입력에서 주어진 트리에서 아무 정점이나 잡아 루트로 만들자.

S 에 속하는 각 정점 v 마다, v 의 부모 정점이 S 에 속하는 경우에만 v 와 v 의 부모 정점을 union하자.

Union된 컴포넌트의 각 크기를 통하여 S 의 연결 강도를 계산할 수 있다.

부분문제 3과 다르게, S 에 속하는 각 정점 v 에 대하여 $\deg(v)$ 개의 간선이 아닌, 단 하나의 간선만을 참조함에 유의하라.

따라서, 각 쿼리당 $\mathcal{O}(K\alpha(N))$ 에 문제를 해결할 수 있다.

Union-find를 사용하지 않고 각 쿼리마다 정점을 깊이 순으로 정렬한 후, 깊은 정점부터 컴포넌트의 크기를 계산해 나가면서 $\mathcal{O}(N + \sum_i K_i \log K_i)$ 에 문제를 해결할 수도 있다. 쿼리를 오프라인으로 받을 경우, 계수 정렬을 사용할 수 있어서 위 알고리즘의 복잡도가 $\mathcal{O}(N + \sum_i K_i)$ 로 최적화된다.

“식사 계획 세우기” 문제 풀이

작성자: 이온조

부분문제 1

N 개의 식당을 방문하는 가능한 모든 식사 계획을 시도해 보고 그 중에 올바른 식사 계획 중 P 가 사전 순으로 가장 앞서는 식사 계획을 계산하면 된다. 시간 복잡도는 $O(N!)$ 이다.

부분문제 2

N 개의 식당 중 임의의 식당 집합 S 를 올바른 식사 계획의 조건을 만족하도록 순서대로 배치할 수 있는지의 여부를 비트마스크를 활용한 동적 계획법으로 계산하면 부분문제 2를 해결할 수 있다. 시간 복잡도는 $O(N \times 2^N)$ 이다.

부분문제 3

크기가 M 인 어떠한 식당 집합 S 에서 가장 많이 판매하는 음식의 종류를 X 라고 하자. 이 때 S 에서 X 를 판매하는 식당의 수를 K_{max} 개라고 하자. 이 때 S 를 올바른 식사 계획의 조건을 만족하도록 순서대로 배치할 수 있다는 것은 $K_{max} \leq \lceil \frac{M}{2} \rceil$ 와 동치이다. 이는 X 를 판매하는 식당을 식사 계획의 홀수 번째에 배치하면 남은 식당들을 적절히 배치하여 올바른 식사 계획을 세우는 것이 항상 가능하기 때문이다.

따라서 탐욕적인 전략을 사용하여 P 가 사전 순으로 가장 앞서도록 앞에서부터 식당을 차례대로 배치하면서, 아직 식사 계획에 배치하지 않은 식당들의 K_{max} 값을 관리하면 올바른 식사 계획의 조건을 항상 만족시키면서 P 를 구할 수 있다. 식당을 앞에서부터 하나씩 배치할 때마다 남은 식당들의 K_{max} 값을 단순히 반복문으로 계산하여 해결하면 시간 복잡도는 $O(N^2)$ 이다.

부분문제 4

부분문제 3의 풀이에서 K_{max} 값을 `std::set` 등의 자료 구조를 사용하여 관리하면 전체 문제를 $O(N \log N)$ 의 시간 복잡도로 해결할 수 있다. 또한 K_{max} 값이 남은 식당 개수의 과반수를 넘을 때만 식당 배치 전략이 달라진다는 점을 이용하여 $O(N)$ 의 시간 복잡도로도 문제를 해결할 수 있다.

“레벨 업” 문제 풀이

작성자: 이민제

부분문제 1

캐릭터를 레벨이 낮은 순서대로 정렬하고, 가장 레벨이 낮은 K 명의 캐릭터의 레벨을 올리는 방식으로 레벨 업 과정을 직접 시뮬레이션할 수 있다. 시간 복잡도는 $O(MN \log N)$ 이다.

부분문제 2

캐릭터를 레벨이 낮은 순서대로 정렬하자. $K = 1$ 인 경우에는 트레이닝이 다음과 같이 이루어짐을 알 수 있다.

- 첫 번째 캐릭터의 레벨이 계속해서 오른다.
- 첫 번째와 두 번째 캐릭터의 레벨이 같아지면, 이후 이 두 캐릭터의 레벨이 번갈아 가며 오른다.
- 첫 번째, 두 번째, 세 번째 캐릭터의 레벨이 같아지면, 이후 이 세 캐릭터의 레벨이 번갈아 가며 오른다.
- 이러한 과정은 모든 캐릭터의 레벨이 같아질 때까지 계속되며, 이후에는 N 개의 캐릭터의 레벨이 번갈아 가며 오른다.

각 $1 \leq i \leq N$ 에 대해, 첫 i 개 캐릭터의 레벨이 같아지는 시점을 누적 합 등을 이용하여 계산할 수 있다. 이 시점이 M 을 초과하지 않으면서 최대가 되도록 하는 i 를 k 라 하자. 이때 $(k+1)$ 번째, $\dots$, N 번째 캐릭터는 레벨이 오르지 않음을 알 수 있다. 한편 1번째 캐릭터와 k 번째 캐릭터의 레벨 차이는 최대 1이므로, 전체 캐릭터의 레벨 총합을 이용하여 각각의 레벨을 구할 수 있다.

시간 복잡도는 $O(N \log N)$ 이다.

부분문제 3

부분문제 3에서는 M 이 크지 않으므로, 각각의 트레이닝을 빠르게 시뮬레이션하는 방식으로 문제를 해결할 수 있다.

캐릭터가 레벨이 낮은 순서대로 정렬되어 있다고 가정하고, K 번째 캐릭터의 레벨을 x 라고 하자. 또, 레벨이 x 미만인 캐릭터의 수를 l , x 이하인 캐릭터의 수를 r 라고 하자. 트레이닝을 한 번 진행하면 캐릭터의 레벨은 다음과 같이 변한다.

- 1번째, $\dots$, l 번째 캐릭터의 레벨이 1 오른다.
- $(l+1)$ 번째, $\dots$, r 번째 캐릭터 중 $(K-l)$ 명의 캐릭터의 레벨이 1 오른다. 편의상 $(l+r-K+1)$ 번째, $\dots$, r 번째 캐릭터의 레벨이 오른다고 하자. 그러면 트레이닝 이후에도 캐릭터들의 레벨이 정렬된 상태를 유지하게 된다.

따라서 정렬된 배열에서 다음과 같은 작업을 효율적으로 수행할 수 있으면 된다.

- 위에서 정의한 x , l , r 을 구한다.

- 구간 $[1, l]$, $[(l + r - K + 1), r]$ 에 1을 더한다.

이는 Lazy Propagation을 사용한 Segment Tree로 빠르게 처리할 수 있다. 시간 복잡도는 $O(M \log N)$ 이다.

부분문제 4

부분문제 2와 마찬가지로, 트레이닝이 진행됨에 따라 캐릭터들의 레벨이 점점 일정해진다는 사실을 관찰할 수 있다. 단, 부분문제 2에서는 레벨이 가장 낮은 캐릭터를 중심으로 레벨이 일정해졌지만, 일반적인 경우에는 레벨이 K 번째로 낮은 캐릭터를 중심으로 레벨이 일정해진다.

맨 처음 상태에서 K 번째로 낮은 레벨을 x 라 할 때, N 명의 캐릭터를 다음과 같이 세 그룹으로 나누자.

- A그룹: 레벨이 x 미만.
- B그룹: 레벨이 x 이상 $x + 1$ 이하.
- C그룹: 레벨이 $x + 1$ 초과.

트레이닝을 진행하면 각 그룹에서 캐릭터들의 레벨은 다음과 같이 변한다.

- A그룹: 모든 캐릭터의 레벨이 1 오른다.
- B그룹: 레벨이 낮은 $(K - |A|)$ 명의 레벨이 1 오른다. (단, $|A|$ 는 그룹 A의 크기.)
- C그룹: 레벨이 오르지 않는다.

트레이닝을 진행하다 보면 A그룹 혹은 C그룹의 캐릭터가 B그룹의 레벨 범위 안으로 들어오는 경우가 발생하는데, 이 경우 해당 캐릭터를 B그룹으로 옮기자. 이때 다음의 사실들을 관찰할 수 있다.

- 레벨이 K 번째로 낮은 캐릭터는 항상 B그룹에 속한다. 즉, 위에서 서술한 각 그룹의 성질은 계속해서 유지된다.
- B그룹 내에서 캐릭터 간의 레벨 차이는 1 이하이다. 따라서 B그룹의 크기와 레벨 범위, 각 레벨에 해당하는 캐릭터의 수를 알면 B그룹 전체의 정보를 얻게 된다. 이 값들을 관리하자.
- A그룹의 캐릭터는 레벨이 높은 순서대로 B그룹으로 들어오며, C그룹의 캐릭터는 레벨이 낮은 순서대로 B그룹으로 들어온다.

A그룹에서 레벨이 가장 높은 캐릭터 또는 C그룹에서 레벨이 가장 낮은 캐릭터가 B그룹으로 들어오는 과정을 시뮬레이션하자. 이 캐릭터가 B그룹으로 들어오는 시점은 앞서 관리한 B그룹의 정보를 바탕으로 이분 탐색(Parametric Search)을 이용하여 계산할 수 있다.

이를 반복하여 각 캐릭터가 B그룹으로 들어오는 시점을 모두 구하자. B 그룹으로 새로운 캐릭터가 들어오는 경우가 총 $O(N)$ 번 발생하므로, 위 과정을 $O(N \log M)$ 시간에 모두 시뮬레이션할 수 있다.

트레이닝이 종료된 시점에서 각 캐릭터가 어느 그룹에 속하는지 안다면, 각각의 레벨 또한 어렵지 않게 구할 수 있다. 총 시간복잡도는 $O(N(\log N + \log M))$ 이다.

“외곽 순환 도로” 문제 풀이

작성자: 구재현

서술의 편의를 위해 풀이에서는 교차로는 정점, 도로는 간선이라고 부른다. $dist(x, y)$ 를 x 와 y 의 최단 경로의 길이라고 정의한다.

부분문제 1

Dijkstra Algorithm을 사용하면 1번 정점에서 시작해서 임의의 정점에 도달하는 데 걸리는 최단 시간을 모두 계산할 수 있다. 문제에서 주어진 그래프는 정점이 N 개이고 간선이 최대 $2N$ 개 이다. Heap 자료구조를 사용한 Dijkstra Algorithm을 사용할 시, $O(N \log N)$ 에 필요한 정보를 모두 계산할 수 있고, 각 쿼리에서는 계산된 정보를 출력해 주면 된다.

시간 복잡도는 $O(N \log N + Q)$ 이다.

부분문제 2

두 정점을 오가는 경로의 경우의 수는 크게 세 가지가 있다.

- 1번 정점을 거쳐서 가는 경우
- 1번 정점을 거치지 않고, 외곽 순환 도로를 시계방향으로 돌아가는 경우
- 1번 정점을 거치지 않고, 외곽 순환 도로를 반시계방향으로 돌아가는 경우

첫 번째 경우는 부분 문제 1과 같이, 1번 정점에서 시작하는 Dijkstra's Algorithm으로 $O(N \log N)$ 에 계산할 수 있다. $dist(s, 1) + dist(1, t) = dist(1, s) + dist(1, t)$ 이기 때문이다. 1번 정점에서 직접 연결된 간선을 사용하면 Dijkstra's Algorithm이 필요 없다고 생각할 수 있지만, 다른 간선으로 외곽 순환 도로에 진입한 후, 외곽 순환 도로를 통해서 목적지에 도착하는 것이 더 효율적일 수 있다.

두 번째와 세 번째 경우는, w_i 배열의 원형 구간 합을 구하는 문제가 된다. 이는 w_i 배열의 부분 합 (Prefix sum) 을 전처리해두면, $O(1)$ 시간에 계산할 수 있다.

시간 복잡도는 $O(N \log N + Q)$ 이다.

부분문제 3

외곽 순환 도로의 가중치가 매우 크기 때문에, 전혀 사용하지 않는 것이 이득이다. 외곽 순환 도로를 모두 무시하면, 문제에서 주어진 그래프는 트리 형태를 띈다.

고로, 문제는 트리 상에서 두 정점 간의 거리를 빠르게 계산하는 것으로 환원된다. 이는 여러 가지 방법으로 해결할 수 있는데, 가장 간단한 방법은 Sparse Table을 사용하여 두 정점의 LCA (Lowest Common Ancestor) 를 계산하고, $dist(1, s) + dist(1, t) - 2 \times dist(1, LCA(s, t))$ 를 출력하는 것이다. $dist(1, *)$ 는 DFS를 통해서 계산할 수 있다.

위 방법을 사용했을 때, 시간 복잡도는 $O((N + Q) \log N)$ 이다.

부분문제 4

어떠한 경로가 외곽 순환 도로를 사용하지 않는 경우는 부분 문제 3과 동일하게 해결할 수 있다.

어떠한 경로가 외곽 순환 도로를 사용하는 경우를 생각해 보자. 외곽 순환 도로 상에 속하는 정점 사이는 비용 없이 오갈 수 있으니, 하나의 정점이라고 생각할 수 있다.

고로, 외곽 순환 도로의 임의의 정점에서 시작해서, 그래프 상의 정점에 도달하는 최단 거리를 계산하면 된다. 외곽 순환 도로 상에 속하는 아무 정점에서 Dijkstra's Algorithm을 사용해도, 결과는 항상 같다. 고로 한 번의 Dijkstra's Algorithm으로 문제를 해결할 수 있다.

시간 복잡도는 $O((N + Q) \log N)$ 이다.

부분문제 5

외곽 순환 도로가 없는 원래 트리를 기준으로, Centroid decomposition을 하자. 현재 보고 있는 Centroid를 c 라고 하자. c 의 자식은 최대 3개이고, 각 자식에 서브트리가 하나씩 있으니, 트리는 최대 3개의 서브트리로 분해됨을 알 수 있다.

어떠한 최적 경로의 형태는 다음 세 가지 중 하나이다:

- Centroid c 를 지난다.
- Centroid c 를 지나지 않으나, c 의 서로 다른 서브트리를 잇는 간선을 사용한다.
- 두 경우 모두 아니다.

첫 번째 경우는 Dijkstra's Algorithm을 사용하여 c 를 시작점으로 한 최단 경로를 계산하면 해결할 수 있다.

두 번째 경우에는 아주 중요한 관찰을 할 수 있다. c 의 서로 다른 서브트리를 잇는 간선은 외곽 순환 도로일 것이고, 그러한 간선은 c 의 자식 개수보다 많지 않다는 것이다. 초기 그래프에서 이 사실은 명백히 참이다. 이 사실이 Subproblem에서도 성립하는 이유는, Subproblem으로 재귀적으로 내려갈 때, 최대 하나의 간선을 추가하여서 초기 그래프와 동일한 환경을 구성해 줄 수 있음을 관찰하면 된다.

고로 c 의 서로 다른 서브트리를 잇는 간선은 최대 3개이다. 이러한 간선을 찾는 것은 단순 DFS로 가능하다. 간선을 찾은 후, 간선의 한쪽 끝에서 Dijkstra's Algorithm을 사용하면 된다.

세 번째 경우가 최적 경로의 형태가 되려면, 쿼리로 주어진 두 정점이 c 가 아니며, 같은 서브트리에 속해야 한다. 이에 해당되는 부분문제가 유일하게 정해지니, 재귀적으로 해결하면 된다.

각 쿼리는 최대 $O(\log N)$ 개의 부분문제에 속하며, 각 레이어에서 가장 계산이 많은 연산은 $O(N \log N)$ 시간에 Dijkstra's Algorithm을 4번 호출하는 것이다. 종합하면, $O(N \log^2 N + Q \log N)$ 시간에 전체 문제가 해결된다.

풀이가 사이클이 감싸고 있는 트리 라는 특수한 구조에 독립적인 편이라, 구현이 일반적인 트리 문제에 비해 크게 어렵지 않다는 점을 참고하면 좋다.

부분문제 6

입력 정점의 차수에 특별한 제한이 없더라도, 차수가 3인 인스턴스로 변환할 수 있다. 어떠한 정점 p 의 자식이 현재 2개고, 내가 새로운 자식 i 를 추가하려고 하는 상황이라고 하자. 단순히 추가할 경우, p 의 차수가 4가 될 수 있지만, 다음과 같은 식으로 추가할 경우 차수가 4인 정점을 만들지 않을 수 있다.

- p' 이라는 새로운 정점을 만든다.
- p 에 마지막으로 추가한 정점을 j 라고 하자. p 와 j 간의 연결을 끊고, p 의 새로운 자식을 p' 으로 두자.
- p' 에 두 자식 i, j 를 이어준다.
- 이후 p 에 대한 포인터를 p' 으로 바꿔준다.

이 외에도 다양한 방법이 있다. $p_i \leq i$ 인 형태로 입력이 주어지기 때문에, 10줄 내외의 짧은 코드 추가로 차수가 3인 인스턴스로 변환할 수 있다. 정점의 개수가 최대 2배가 되지만, 복잡도에는 영향이 없다.

시간 복잡도는 $O(N \log^2 N + Q \log N)$ 이다.